

 ggerganov / llama.cpp



<> Code 302 Issues Pull requests 236 Discussions Actions Projects 8 Wiki Security 1 

llama.cpp / grammars / 

Add file ▾

daghanerdonmez readme : fix typo [no ci] (#8389)  9beb2dd · 5 days ago 

Name	Name	Last commit date
 ..		
 README.md	readme : fix typo [no ci] (#8389)	5 days ago
 arithmetic.gbnf	llama : add grammar-based sampling (#17...	last year
 c.gbnf	examples : add C grammar (#2357)	last year
 chess.gbnf	llama : add grammar-based sampling (#17...	last year
 japanese.gbnf	llama : add grammar-based sampling (#17...	last year
 json.gbnf	json: refine constraint for whitespace to av...	last month
 json_arr.gbnf	json: refine constraint for whitespace to av...	last month
 list.gbnf	llama : add grammar-based sampling (#17...	last year

README.md



GBNF Guide

GBNF (GGML BNF) is a format for defining [formal grammars](#) to constrain model outputs in `llama.cpp`. For example, you can use it to force the model to generate valid JSON, or speak only in emojis. GBNF grammars are supported in various ways in `examples/main` and `examples/server`.

Background

[Backus-Naur Form \(BNF\)](#) is a notation for describing the syntax of formal languages like programming languages, file formats, and protocols. GBNF is an extension of BNF that primarily adds a few modern regex-like features.

Basics

In GBNF, we define *production rules* that specify how a *non-terminal* (rule name) can be replaced with sequences of *terminals* (characters, specifically Unicode [code points](#)) and other non-terminals. The basic format of a production rule is `nonterminal ::= sequence...`.

Example

Before going deeper, let's look at some of the features demonstrated in `grammars/chess.gbnf`, a small chess notation grammar:

```
# `root` specifies the pattern for the overall output
root ::= (
    # it must start with the characters "1. " followed by a sequence
    # of characters that match the `move` rule, followed by a space, followed
    # by another move, and then a newline
    "1. " move " " move "\n"
```



```
# it's followed by one or more subsequent moves, numbered with one or two digits
([1-9] [0-9]? ". " move " " move "\n")+
)

# `move` is an abstract representation, which can be a pawn, nonpawn, or castle.
# The `[+#]` denotes the possibility of checking or mate signs after moves
move ::= (pawn | nonpawn | castle) [+#]?

pawn ::= ...
nonpawn ::= ...
castle ::= ...
```

Non-Terminals and Terminals

Non-terminal symbols (rule names) stand for a pattern of terminals and other non-terminals. They are required to be a dashed lowercase word, like `move`, `castle`, or `check-mate`.

Terminals are actual characters ([code points](#)). They can be specified as a sequence like `"1"` or `"0-0"` or as ranges like `[1-9]` or `[NBKQR]`.

Characters and character ranges

Terminals support the full range of Unicode. Unicode characters can be specified directly in the grammar, for example `hiragana ::= [あ-ぢ]`, or with escapes: 8-bit (`\xxx`), 16-bit (`\uxxxx`) or 32-bit (`\uxxxxxxxx`).

Character ranges can be negated with `^`:

```
single-line ::= [^\n]+ "\n"
```



Sequences and Alternatives

The order of symbols in a sequence matters. For example, in `"1. " move " " move "\n"`, the `"1. "` must come before the first `move`, etc.

Alternatives, denoted by `|`, give different sequences that are acceptable. For example, in `move ::= pawn | nonpawn | castle`, `move` can be a `pawn move`, a `nonpawn move`, or a `castle`.

Parentheses `()` can be used to group sequences, which allows for embedding alternatives in a larger rule or applying repetition and optional symbols (below) to a sequence.

Repetition and Optional Symbols

- `*` after a symbol or sequence means that it can be repeated zero or more times (equivalent to `{0,}`).
- `+` denotes that the symbol or sequence should appear one or more times (equivalent to `{1,}`).
- `?` makes the preceding symbol or sequence optional (equivalent to `{0,1}`).
- `{m}` repeats the precedent symbol or sequence exactly `m` times
- `{m,}` repeats the precedent symbol or sequence at least `m` times
- `{m,n}` repeats the precedent symbol or sequence at between `m` and `n` times (included)
- `{0,n}` repeats the precedent symbol or sequence at most `n` times (included)

Comments and newlines

Comments can be specified with `#`:

```
# defines optional whitespace
ws ::= [ \t\n]+
```



Newlines are allowed between rules and between symbols or sequences nested inside parentheses. Additionally, a newline after an alternate marker `|` will continue the current rule, even outside of parentheses.

The root rule

In a full grammar, the `root` rule always defines the starting point of the grammar. In other words, it specifies what the entire output must match.

```
# a grammar for lists
root ::= (" " item)+
item ::= [^\n]+ "\n"
```



Next steps

This guide provides a brief overview. Check out the GBNF files in this directory (`grammars/`) for examples of full grammars. You can try them out with:

```
./llama-cli -m <model> --grammar-file grammars/some-grammar.gbnf -p 'Some prompt'
```



`llama.cpp` can also convert JSON schemas to grammars either ahead of time or at each request, see below.

Troubleshooting

Grammars currently have performance gotchas (see [#4218](#)).

Efficient optional repetitions

A common pattern is to allow repetitions of a pattern `x` up to `N` times.

While semantically correct, the syntax `x? x? x? ... x?` (with `N` repetitions) may result in extremely slow sampling. Instead, you can write `x{0,N}` (or `(x (x (x ... (x)?...)?))?` w/ `N`-deep nesting in earlier `llama.cpp` versions).

Using GBNF grammars

You can use GBNF grammars:

- In [llama-server](#)'s completion endpoints, passed as the `grammar` body field
- In [llama-cli](#), passed as the `--grammar` & `--grammar-file` flags
- With [llama-gbnf-validator](#) tool, to test them against strings.

JSON Schemas → GBNF

`llama.cpp` supports converting a subset of <https://json-schema.org/> to GBNF grammars:

- In [llama-server](#):
 - For any completion endpoints, passed as the `json_schema` body field
 - For the `/chat/completions` endpoint, passed inside the `result_format` body field (e.g. `{"type": "json_object", "schema": {"items": {}}}`)
- In [llama-cli](#), passed as the `--json` / `-j` flag
- To convert to a grammar ahead of time:
 - in CLI, with [examples/json_schema_to_grammar.py](#)
 - in JavaScript with [json-schema-to-grammar.mjs](#) (this is used by the [server](#)'s Web UI)

Take a look at [tests](#) to see which features are likely supported (you'll also find usage examples in [#5978](#), [#6659](#) & [#6555](#)).

```
llama-cli \
-hfr bartowski/Phi-3-medium-128k-instruct-GGUF \
-hff Phi-3-medium-128k-instruct-Q8_0.gguf \
-j '{
  "type": "array",
  "items": {
    "type": "object",
```



```

    "properties": {
        "name": {
            "type": "string",
            "minLength": 1,
            "maxLength": 100
        },
        "age": {
            "type": "integer",
            "minimum": 0,
            "maximum": 150
        }
    },
    "required": ["name", "age"],
    "additionalProperties": false
},
"minItems": 10,
"maxItems": 100
}' \
-p 'Generate a {name, age}[] JSON array with famous actors of all ages.'

```

► Show grammar

Here is also a list of known limitations (contributions welcome):

- `additionalProperties` defaults to `false` (produces faster grammars + reduces hallucinations).
- `"additionalProperties": true` may produce keys that contain unescaped newlines.
- Unsupported features are skipped silently. It is currently advised to use the command-line Python converter (see above) to see any warnings, and to inspect the resulting grammar / test it w/ [llama-gbnf-validator](#).
- Can't mix `properties` w/ `anyOf` / `oneOf` in the same type ([#7703](#))
- [prefixItems](#) is broken (but [items](#) works)
- `minimum`, `exclusiveMinimum`, `maximum`, `exclusiveMaximum`: only supported for `"type": "integer"` for now, not `number`
- Nested `$ref`s are broken ([#8073](#))
- [patterns](#) must start with `^` and end with `$`
- Remote `$ref`s not supported in the C++ version (Python & JavaScript versions fetch https refs)
- string [formats](#) lack `uri`, `email`
- No [patternProperties](#)

And a non-exhaustive list of other unsupported features that are unlikely to be implemented (hard and/or too slow to support w/ stateless grammars):

- [uniqueItems](#)
- [contains](#) / `minContains`
- `$anchor` (cf. [dereferencing](#))
- [not](#)
- [Conditionals](#) `if` / `then` / `else` / `dependentSchemas`

A word about additionalProperties

⚠ Warning

The JSON schemas spec states `object`s accept [additional properties](#) by default. Since this is slow and seems prone to hallucinations, we default to no additional properties. You can set `"additionalProperties": true` in the the schema of any object to explicitly allow additional properties.

If you're using [Pydantic](#) to generate schemas, you can enable additional properties with the `extra` config on each model class:

```

# pip install pydantic
import json
from typing import Annotated, List
from pydantic import BaseModel, Extra, Field
class QAPair(BaseModel):
    class Config:

```



```
        extra = 'allow' # triggers additionalProperties: true in the JSON schema
question: str
concise_answer: str
justification: str

class Summary(BaseModel):
    class Config:
        extra = 'allow'
    key_facts: List[Annotated[str, Field(pattern='- .{5,}')] ]
    question_answers: List[Annotated[List[QAPair], Field(min_items=5)]]

print(json.dumps(Summary.model_json_schema(), indent=2))
```

► Show JSON schema & grammar

If you're using [Zod](#), you can make your objects to explicitly allow extra properties w/ `nonstrict()` / `passthrough()` (or explicitly no extra props w/ `z.object(...).strict()` or `z.strictObject(...)`) but note that [zod-to-json-schema](#) currently always sets `"additionalProperties": false` anyway.

```
import { z } from 'zod';
import { zodToJsonSchema } from 'zod-to-json-schema';

const Foo = z.object({
  age: z.number().positive(),
  email: z.string().email(),
}).strict();

console.log(zodToJsonSchema(Foo));
```

► Show JSON schema & grammar

